



MDN docs turns 15

This popular web platform documentation sites turned 15 this year, here is a look at the journey. <http://dgit.in/sep20-29>



Information is getting massive

Researchers estimate that by 2245, information could make up half the mass in the entire world. <http://dgit.in/sep20-30>

garbage collection is being handled properly or not. Modern programming languages often have inherent garbage collection modules so this may not be a concern but there are plenty of languages where you still have to work on garbage collection yourself. Profilers can also track the processor and memory usage so you can find out if your program is killing off others or taking a ridiculous amount of time to be initiated after being dismissed.

Let's look at optimising Android apps. The program recommended by Google to do this is `simpleperf`. It's a native profiling tool for Android which uses the same options as the linux-tools `perf` but is more tuned for Android. `Simpleperf` is also part of the Android Open Source Project (AOSP). `Simpleperf` relies on the Performance Monitoring Unit (PMU) which is a hardware counter in your smartphone which keeps track of events like how many CPU cycles have happened, the number of instructions that have been executed, if there have been any cache misses, etc.

You'll end up using three simple commands when using `Simpleperf` and they are `stat`, `record` and `report`.

stat - provides a summary of how many events have occurred in the profiled process within a certain time period.

record - records metrics of the process that is being profiled over a certain time period

report - once a record command is run, it generates a file called `perf`. data which is read and a report is outputted onto the screen with details showing where time was spent

We start off by listing all the events that are available on the device that we are testing. To do this type: `simpleperf list`

You should get a sample output as follows:

```
List of hw-cache events:
branch-loads
...
```

```
List of hardware events:
cpu-cycles
instructions
...
```

```
List of software events:
cpu-clock
task-clock
...
```

Using the `stat` command without selecting any specific events, you can get all the events listed for it by default. Here's an example:

```
# Stat using default events
(cpu-cycles,instructions,...),
and monitor
# process 7394 for 10 seconds.
$simpleperf stat -p 7394
--duration 10
Performance counter statistics:

1,320,496,145  cpu-cycles
# 0.131736 GHz
(100%)
510,426,028  instructions
# 2.587047 cycles per instruction (100%)
4,692,338  branch-misses # 468.118 K/sec
(100%)
886.008130(ms)  task-clock
# 0.088390 cpus used
(100%)
753  context-switches
# 75.121 /sec
(100%)
870  page-faults
# 86.793 /sec
(100%)

Total test time: 10.023829
seconds.
```

You can select a particular event using the `-e` flag.

```
# Stat event cpu-cycles.
$simpleperf stat -e cpu-cycles -p 11904 --duration 10

# Stat event cache-references
and cache-misses.
$simpleperf stat -e cache-references,cache-misses -p
```

```
11904 --duration 10
```

When you are monitoring, you need to keep in mind that the PMU has a limited number of hardware counters, so not everything can be monitored all the time. If you give way too many metrics to be monitored, then the hardware counters will be used on a rotational basis. If this happens, you will see it in the report at the end of each line in the form of a percentage value.

```
# Stat using event cache-references, cache-references:u,...
$simpleperf stat -p 7394 -e cache-references,cache-references:u,cache-references:k,cache-misses,cache-misses:u,cache-misses:k,instructions
--duration 1
Performance counter statistics:

4,331,018  cache-references
# 4.861 M/sec (66%)
3,064,089  cache-references:u
# 3.439 M/sec (66%)
1,364,959  cache-references:k
# 1.532 M/sec (66%)
91,721  cache-misses
# 102.918 K/sec (66%)
45,735  cache-misses:u
# 51.327 K/sec (66%)
38,447  cache-misses:k
# 43.131 K/sec (66%)
9,688,515  instructions
# 10.561 M/sec (66%)

Total test time: 1.026802
seconds.
```

This tells us that the event was only monitored for 66 per cent of the time since there weren't enough hardware counters. There will also be instances where you need to monitor some events at the same time, in such cases you can group the events together. We then have to move on to figuring out which target to monitor. If we had to monitor any process or threads to monitor,